

DAY 5 — BACKEND TASKS

Nandini Bhimineni

Yoganandini9@gmail.com

TASK 1 — Create Product API Structure

Create a new backend project folder named product-api.

Create the following folders:

- models
- routes
- controllers
- config

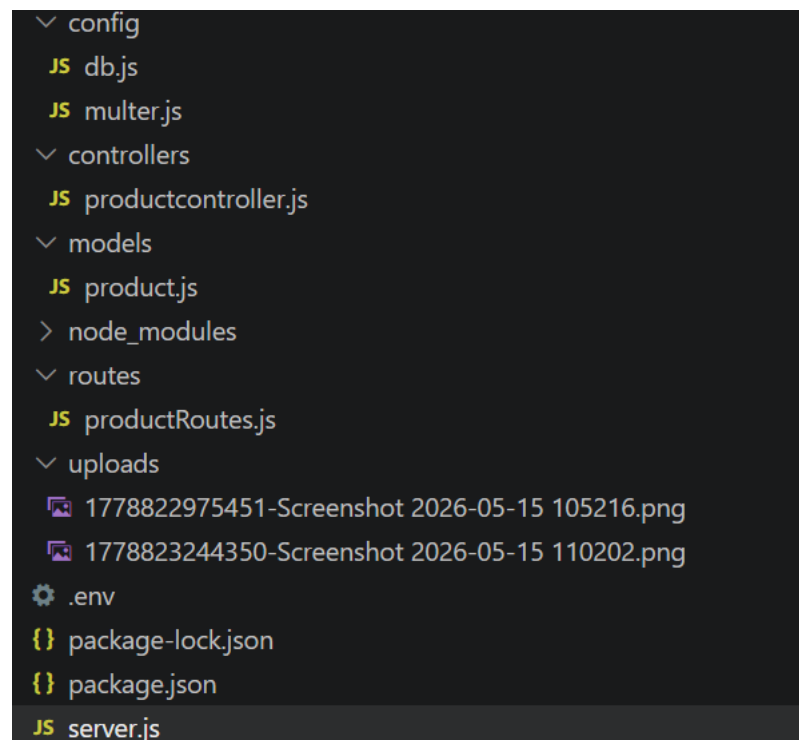
Create these files:

- server.js
- .env
- package.json

Initialize the Node.js project and install:

express, mongoose, dotenv, nodemon

Setup a basic Express server and connect MongoDB.



```
JS server.js • 1778823244350-Screenshot 2026-05-15 110202.png JS multer.js JS
JS server.js > ...
1  const express = require("express");
2  const dotenv = require("dotenv");
3
4  const connectDB = require("../config/db");
5
6  dotenv.config();
7
8  // Connect MongoDB
9  connectDB();
10
11  const app = express();
12
13  // Middleware
14  app.use(express.json());
15
16  // Make uploads folder publicly accessible (for images)
17  app.use("/uploads", express.static("uploads"));
18
19
20  // Routes
21  const productRoutes = require("../routes/productRoutes");
22  app.use("/api/products", productRoutes);
23
24  // Home route
25  app.get("/", (req, res) => {
26    | res.send("Server Running Successfully");
27  });
28
29  // Start server
30  const PORT = process.env.PORT || 5000;
31
32  app.listen(PORT, () => {
33    | console.log(`Server running on port ${PORT}`);
34  });
```

.env

```
gear .env
1  PORT=5000
2  MONGO_URI=mongodb://localhost:27017/wardrobe
```

productRoutes.js

```
routes > JS productRoutes.js > ...
1  const express = require("express");
2  const router = express.Router();
3
4  const {
5    | addProduct,
6    | getProducts,
7  } = require("../controllers/productController");
8
9  const upload = require("../config/multer");
10
11  router.post("/add", upload.single("image"), addProduct);
12
13  router.get("/", getProducts);
14
15  module.exports = router;
```

Product.js

```
models > JS product.js > [C] productSchema
1  const mongoose = require("mongoose");
2
3  const productSchema = new mongoose.Schema({
4    productName: {
5      type: String,
6      required: true,
7    },
8
9    brand: {
10     type: String,
11   },
12
13   category: {
14     type: String,
15   },
16
17   price: {
18     type: Number,
19     required: true,
20   },
21
22   stock: {
23     type: Number,
24     default: 0,
25   },
26
27   image: {
28     type: String,
29   },
30 });
31
32 module.exports = mongoose.model("Product", productSchema);
```

Productcontroller.js

```
controllers > JS productcontroller.js > ...
1  const Product = require("../models/product");
2
3  const addProduct = async (req, res) => {
4    try {
5      const product = new Product({
6        productName: req.body.productName,
7        brand: req.body.brand,
8        category: req.body.category,
9        price: req.body.price,
10       stock: req.body.stock,
11       image: req.file ? req.file.filename : "",
12     });
13
14     await product.save();
15
16     res.status(201).json({
17       message: "Product Added Successfully",
18       product,
19     });
20   } catch (error) {
21     res.status(500).json({
22       message: error.message,
23     });
24   }
25 };
26
27 const getProducts = async (req, res) => {
28   try {
29     const products = await Product.find();
30
31     res.json(products);
32   } catch (error) {
33     res.status(500).json({
34       message: error.message,
35     });
36   }
37 };
38
39 module.exports = {
40   addProduct,
41   getProducts,
42 };
```

Db.js

```
config > JS dbjs > ...
1  const mongoose = require("mongoose");
2
3  const connectDB = async () => {
4    try {
5      await mongoose.connect(process.env.MONGO_URI);
6
7      console.log("MongoDB Connected");
8    } catch (error) {
9      console.log(error);
10
11      process.exit(1);
12    }
13  };
14
15  module.exports = connectDB;
```

Multer.js

```
config > JS multer.js > ...
1  const multer = require("multer");
2
3  const storage = multer.diskStorage({
4    destination: function (req, file, cb) {
5      cb(null, "uploads/");
6    },
7
8    filename: function (req, file, cb) {
9      cb(null, Date.now() + "-" + file.originalname);
10    },
11  });
12
13  const upload = multer({ storage });
14
15  module.exports = upload;
```

TASK 2 — Build Product Details API

Create a Product model with the following fields:

- productName
- price
- brand
- category
- stock

Create:

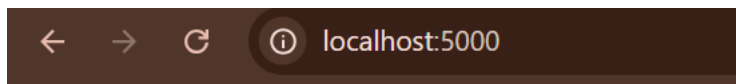
- Product model
- Product controller
- Product routes

Create POST API:

/api/products/add

The API should:

- Accept product data
- Store data in MongoDB
- Return success message

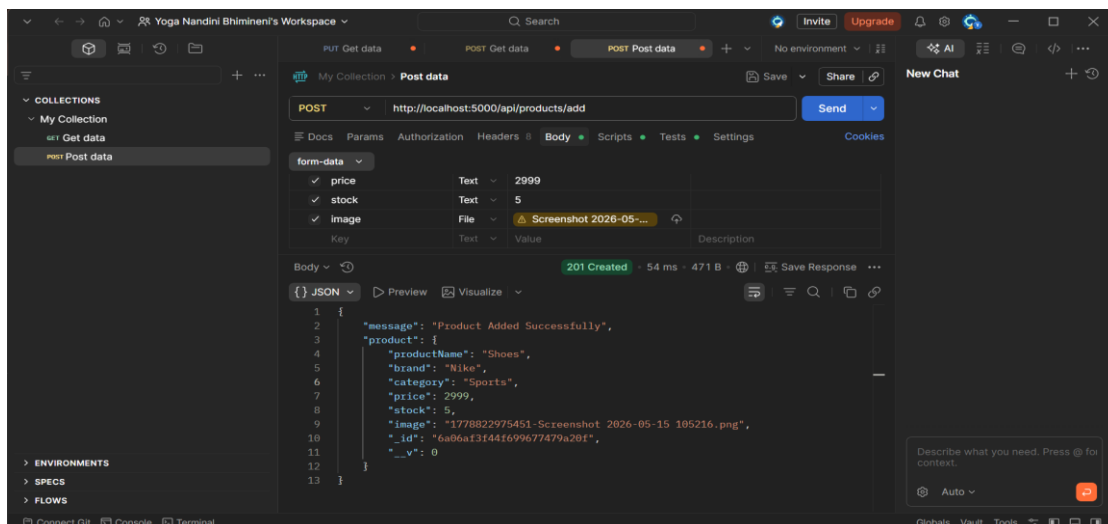


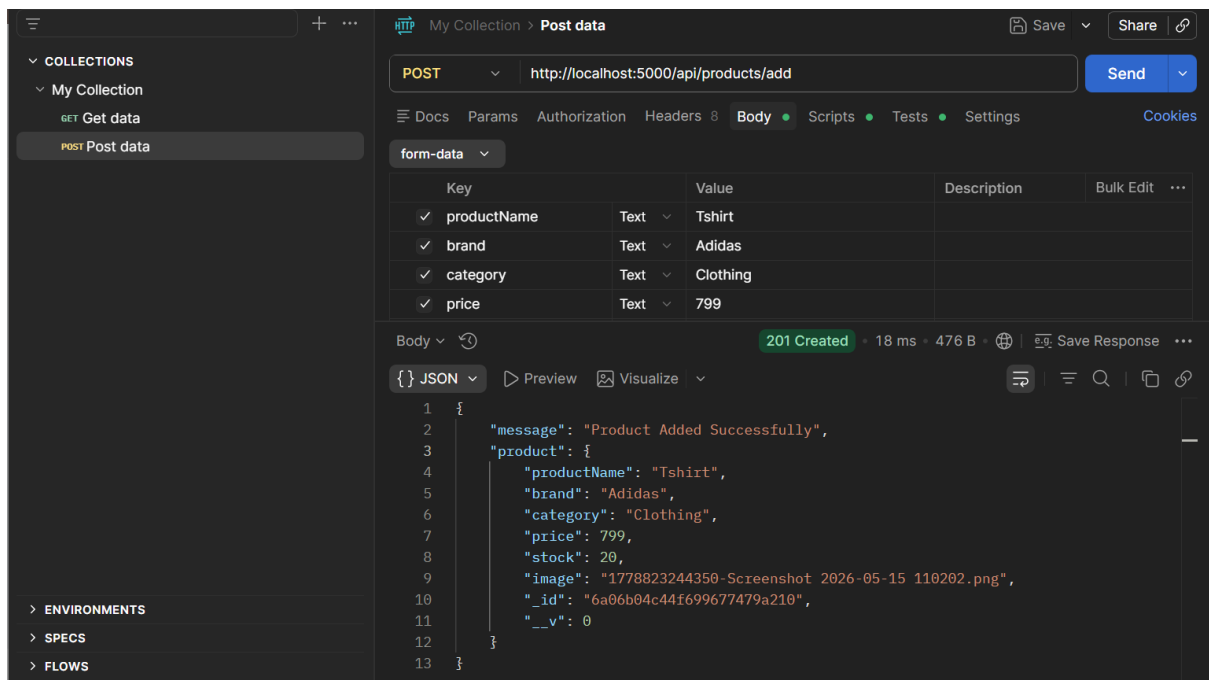
Server Running Successfully

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\yogan\Downloads\Raritone\wardrobe_API> npm install multer
27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
● PS C:\Users\yogan\Downloads\Raritone\wardrobe_API> node server.js
○ PS C:\Users\yogan\Downloads\Raritone\wardrobe_API> node server.js
Server is Running on Port 5000
Connected to MongoDB
```





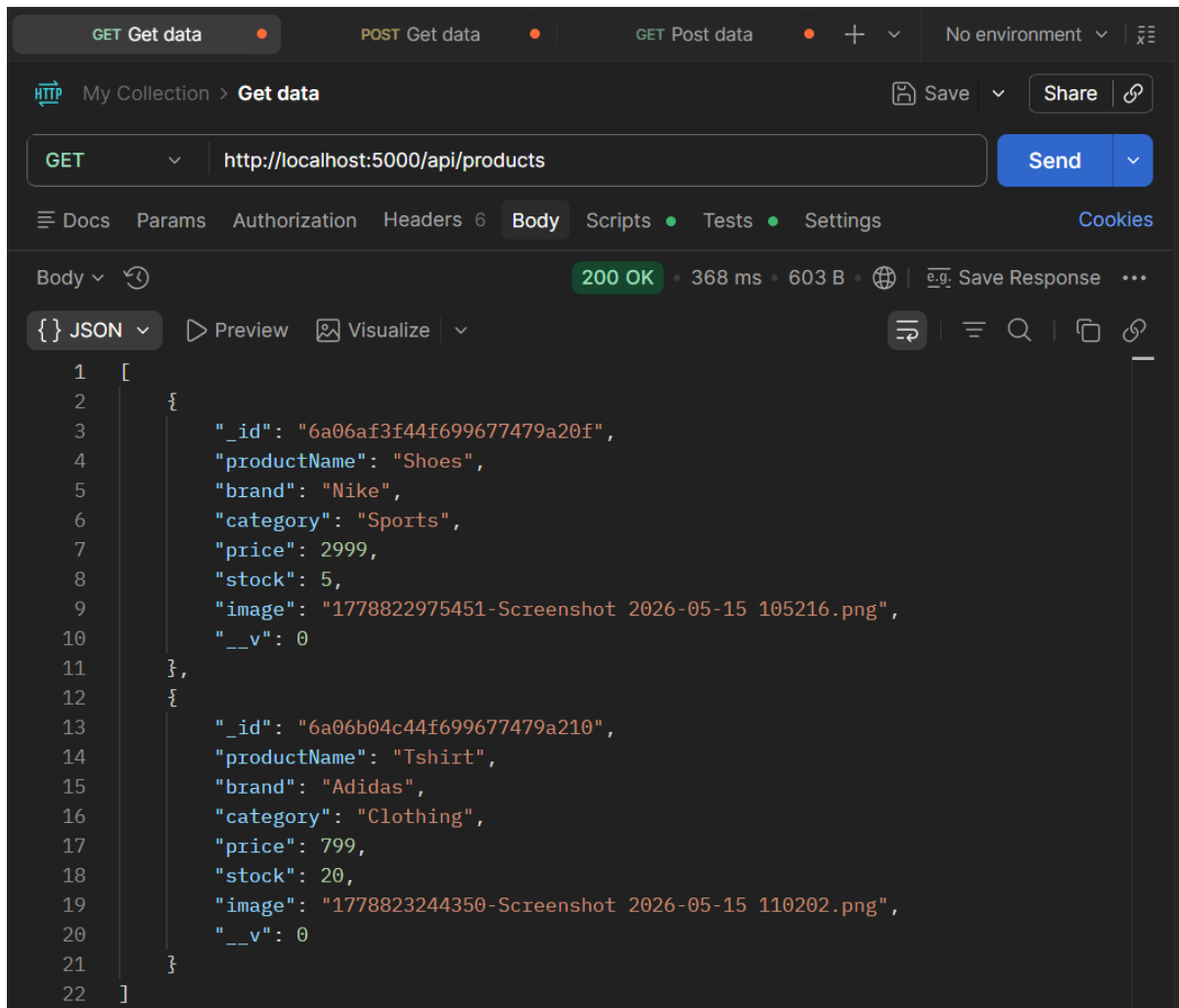
TASK 3 — Create Get Products API

Create GET API:

/api/products

The API should:

- Fetch all products from MongoDB
- Return products in JSON format



TASK 1 — Create Product API Structure

I created a backend project named **product-api** and organized it using a proper folder structure for better maintainability and scalability.

I created required folders such as models, routes, controllers, and config to follow MVC architecture.

Then I initialized the Node.js project and installed required dependencies like Express, Mongoose, Dotenv, and Nodemon.

After setup, I created the main server file and configured a basic Express server. I also connected the application to MongoDB using Mongoose. The database connection was handled securely using environment variables stored in the .env file.

Finally, I tested the server and confirmed that both Express server and MongoDB connection were working successfully.

MULTER IMPLEMENTATION (File Upload Feature)

In this project, I also implemented **Multer** for handling image uploads.

Multer is a middleware used in Node.js to handle file uploads such as images.

In this project, it was used to upload product images along with product details.

How Multer Works:

- The API accepts data using **form-data** instead of JSON
- Product details are sent along with an image file
- Multer processes the uploaded file on the server
- The image is stored in a local folder called **uploads**
- A unique filename is generated for each uploaded image
- Only the image filename is stored in MongoDB along with product data

Testing Multer:

The file upload functionality was tested using Postman:

- Request type used: POST
- Data sent using form-data
- Text fields included product details
- Image uploaded as file input
- Image successfully stored in uploads folder
- Product data saved in MongoDB along with image name

TASK 2 — Build Product Details API (POST API)

In this task, I created a Product API using MVC architecture.

I defined a Product model with fields such as product name, price, brand, category, and stock.

Then I created a controller to handle product creation logic. It receives product data from the client, processes it, and stores it in MongoDB.

After that, I created routes to handle API requests and connected them to the controller.

Finally, I implemented a POST API that accepts product details and stores them in MongoDB. After successful insertion, a success response is returned.

TASK 3 — Create Get Products API (GET API)

In this task, I created a GET API to retrieve all products stored in MongoDB.

The API fetches all product records from the database and returns them in JSON format.

This allows users to view all stored product data easily.

FINAL SUMMARY

In this project, I successfully built a complete backend system using Node.js, Express, and MongoDB.

I followed MVC architecture to structure the project properly and implemented APIs for creating and fetching products.

I also integrated Multer to handle image uploads, where product images are stored in a local folder and linked with MongoDB data.

All APIs were tested using Postman and worked successfully.